MIT 8103 Advanced Database Systems

Practical Database Portfolio – LMS Documentation

Student	Ikenna Nwosu	**Student ID**	202609417
Programme	Master of Information Technology	**Session**	2026/2027 First Semester CA
Case study	Kasuwa Marketplace (simulated e-commerce)	**GitHub**	github.com/<your-username>/advanced-database-portfolio

1. Selected organisation

Kasuwa Marketplace ("kasuwa" is Hausa for market) is a simulated Nigerian multi-vendor e-commerce platform. Independent sellers list electronics, fashion, home, grocery and beauty products; customers in all 36 states and the FCT order online; payments are taken by card, bank transfer, USSD, wallet or pay-on-delivery; third-party couriers deliver. **Users:** customers, sellers, operations staff, finance, management and the platform's web/mobile application. **Major data:** customers and addresses, sellers, product catalogue and category tree, inventory, orders and order lines, payments, deliveries, reviews, stock movements. **Key operations:** place order, update stock, search catalogue, track delivery, review products, and report sales by region and category. A reproducible synthetic data set (50,000 customers, 5,000 products, 300,000 orders, 600,153 order lines) is used in all five activities.

2. Portfolio activities

Portfolio 1 – Design and modelling. An ER diagram in crow's-foot notation was converted to 14 PostgreSQL tables. Normalisation was traced from an un-normalised order form through 1NF, 2NF and 3NF to BCNF. Constraints include surrogate and composite keys, a partial unique index (one default address per customer), CHECKs (Nigerian phone format, stock ≥ 0, rating 1–5) and chosen FK actions. Ten deliberately invalid operations were all rejected by the DBMS. Two design decisions are defended: order_item keeps the price at time of sale, and order totals are derived by a view rather than stored.

Portfolio 2 – Query processing and optimisation. Five workload queries were measured with EXPLAIN (ANALYZE, BUFFERS) in three phases (median of 7 warm runs; all 15 plans saved). Round 1 added targeted indexes and a materialised view; Q1 fell from 29.6 ms to 0.17 ms and Q3 from 1.3 s to 0.04 ms, but Q2 and Q5 got slower. Reading the plans showed why (a hash join over all 600k order lines; default random_page_cost = 4 over-pricing index probes on SSD). Round 2 tuned the cost model and pre-aggregated daily product sales, giving final speed-ups of 3.7× to ~42,000×, with rewritten queries verified to return identical rows.

Portfolio 3 – Transactions and concurrency. Checkout was implemented as one atomic transaction and demonstrated with COMMIT, ROLLBACK, an automatic abort on a CHECK violation and a SAVEPOINT that keeps the in-stock line. Seven scenarios with real concurrent sessions reproduced a lost update, then fixed it three ways (SELECT … FOR UPDATE, REPEATABLE READ with retry on SQLSTATE 40001, optimistic version column); showed a non-repeatable read; provoked and fixed a deadlock (40P01) by ordering locks; and ran a 60-buyer flash sale on 20 units: the naive method oversold 40 units, the atomic conditional UPDATE oversold none.

Portfolio 4 – NoSQL and advanced data models. Redis 7 holds the key-value workload (sessions with TTL, carts, a cache-aside product cache, a flash-sale counter decremented by an atomic Lua script – exactly 20 of 60 buyers succeeded – and a sorted-set bestseller leaderboard, 0.19 ms vs 72 ms in SQL). The product catalogue, whose attributes differ by category, was modelled as documents in PostgreSQL JSONB with bounded embedded reviews; a GIN index cut a containment query from 3.3 ms to 0.40 ms. The same documents were queried through the MongoDB API (find, aggregation pipeline, $unwind, $push/$slice) and cross-checked as identical. Orders and payments deliberately stay relational.

Portfolio 5 – Distributed and cloud databases. A four-server PostgreSQL 16 lab was built: primary, streaming-replication hot standby, and two shards partitioned by geopolitical zone via postgres_fdw. Experiments showed partition pruning, scatter-gather without the shard key, aggregate push-down (230 ms → 56 ms), partial availability when a shard fails, ~1 ms replication lag, the latency cost of synchronous commit, a CAP trade-off (sync commit blocks with the standby down), and a failover in ~100 ms with all 923 committed probe rows preserved. A managed-cloud deployment (multi-AZ HA, read replica, managed Redis, PITR backups, NDPA 2023 data location) is proposed.

3. Selected evidence

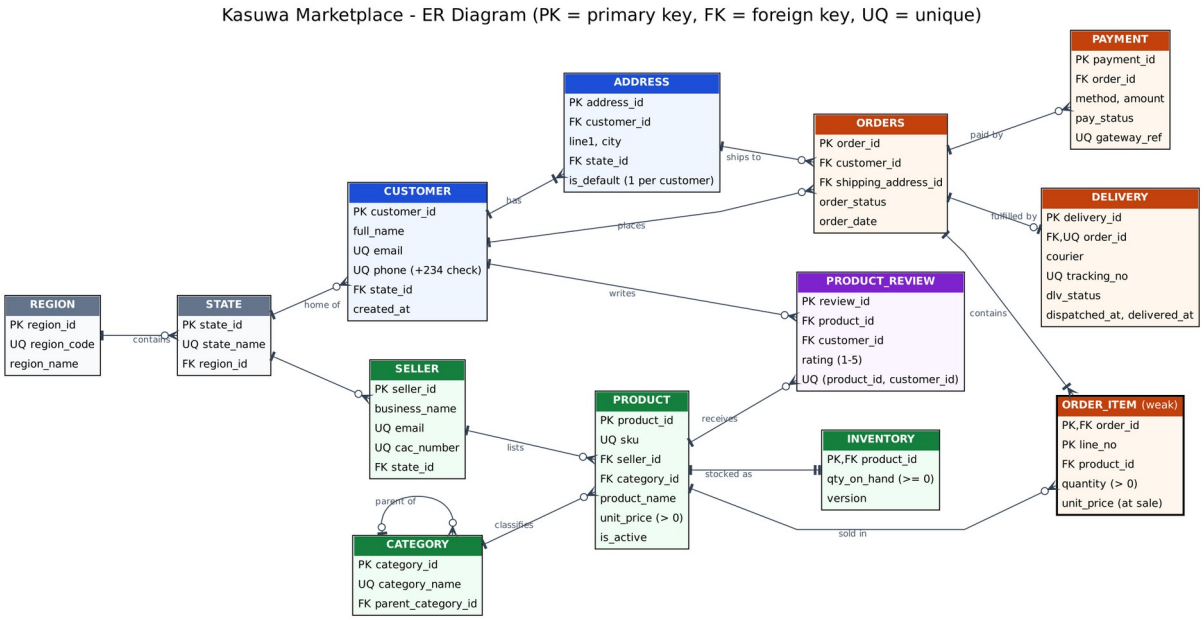


Figure 1. Kasuwa ER diagram (Portfolio 1; source erd.dot in the repository)

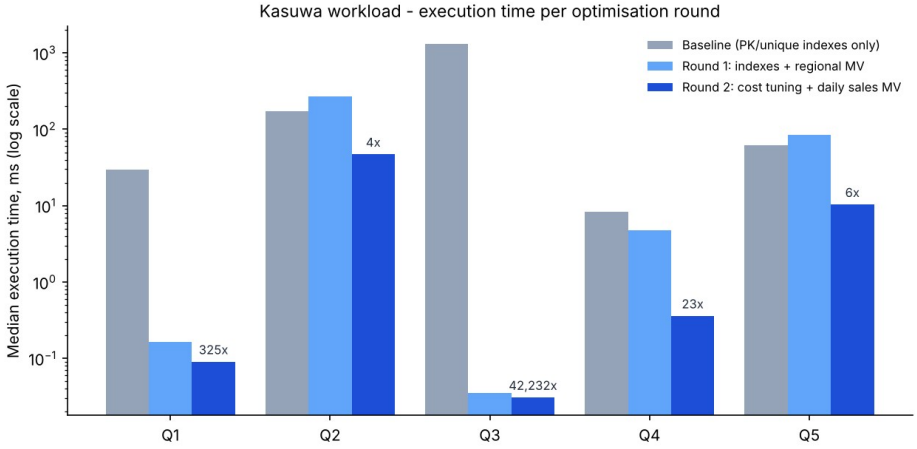


Figure 2. Median execution time per optimisation round, log scale (Portfolio 2)

```
python3 concurrency_demo.py  - outcomes of scenarios S1-S7

S1  LOST UPDATE - two buyers, READ COMMITTED, application computes new stock (read-modify-write)
RESULT: units sold = 2, stock should be 3, stock recorded = 4  => LOST UPDATE: one sale vanished from stock
S2  FIX A - PESSIMISTIC LOCKING with SELECT ... FOR UPDATE
[  1.100s] T2     (waited 1.01s for T1's row lock, then read the COMMITTED value 4)
RESULT: units sold = 2, stock recorded = 3 (expected 3) => CORRECT - T2 was serialised behind T1
S3  FIX B - REPEATABLE READ: concurrent update detected, second transaction must retry
[  1.132s] T2     !! SQLSTATE 40001 ERROR:  could not serialize access due to concurrent update
RESULT: units sold = 2, stock recorded = 3 (expected 3) => CORRECT after one retry
S4  FIX C - OPTIMISTIC LOCKING: UPDATE ... WHERE version = <version read>
RESULT: units sold = 2, stock recorded = 3 (expected 3) => CORRECT - no locks held between read and write
S5  ISOLATION LEVELS: finance report reads a price twice while a seller changes it
RESULT (READ COMMITTED): first read 50000.00, second read 55000.00 in the SAME transaction => NON-REPEATABLE READ occurred
RESULT (REPEATABLE READ): first read 55000.00, second read 55000.00 in the SAME transaction => stable snapshot, no anomaly
S6  DEADLOCK - two baskets containing the same two products, locked in opposite order
[  2.270s] T1     !! SQLSTATE 40P01 ERROR:  deadlock detected
RESULT: T1 DEADLOCK VICTIM - rolled back by PostgreSQL (app would retry); T2 committed
RESULT: T1 committed; T2 committed
S7  FLASH SALE STRESS TEST - 60 concurrent buyers, 20 units in stock, 1 unit each
NAIVE: SELECT stock, check in app, UPDATE stock = value_read - 1
   buyers told 'sold' = 60, rejected = 0, sale records = 60, stock left = 19, stock should be = -40, oversold units = 40  (0.55s)
ATOMIC: UPDATE ... SET qty = qty - 1 WHERE qty >= 1 RETURNING (row lock + recheck)
   buyers told 'sold' = 20, rejected = 40, sale records = 20, stock left = 0, stock should be = 0, oversold units = 0  (0.57s)
```

Figure 3. Outcomes of concurrency scenarios S1–S7 (Portfolio 3)

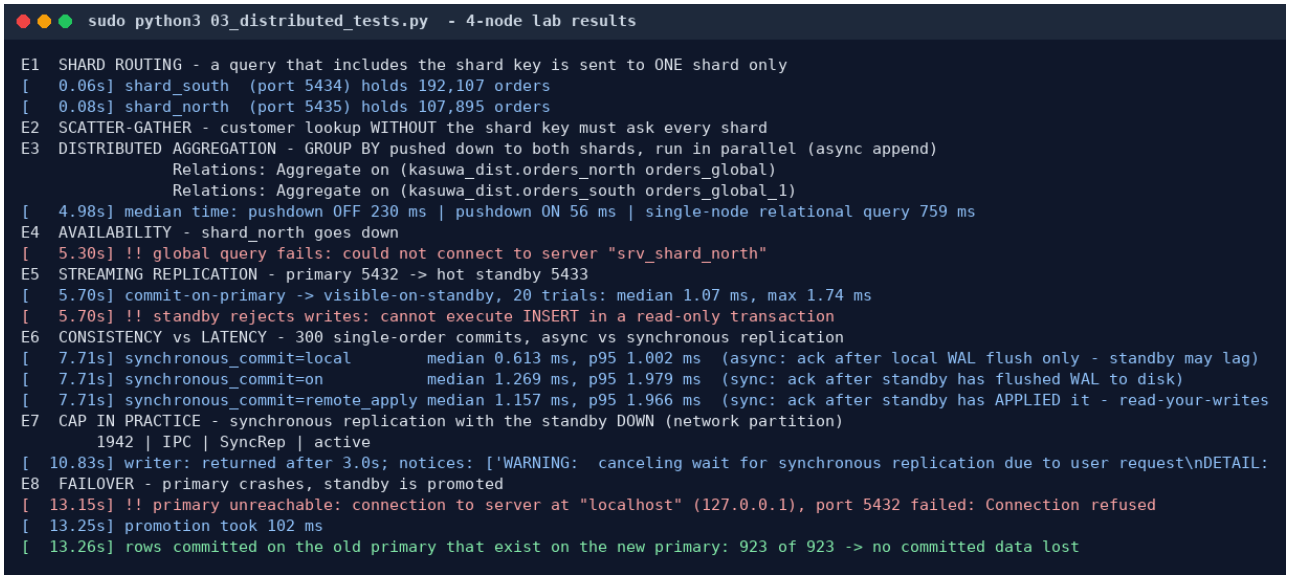


Figure 4. Distributed experiments E1–E8 on the four-node lab (Portfolio 5)

All raw outputs, execution plans, scripts and per-portfolio READMEs are in the GitHub repository; each README states how the activity was implemented and tested and how to reproduce it.

4. Individual technical reflection

Design choices. I chose an e-commerce case because it exercises every topic in the course: strict relational integrity for money and stock, a catalogue whose attributes vary by category, high-frequency key-value data such as carts, and a natural shard key in Nigeria's geopolitical zones. I kept derived data (order totals) out of the base tables and only reintroduced it as materialised views once a measured report needed it, and I kept orders and payments relational even after adding Redis and documents, because they need multi-row ACID transactions.

Challenges. Three results surprised me. My first optimisation round made Q2 and Q5 slower; I had to compare estimated and actual rows in the plans to see that the planner's default cost model, not the missing index, was the problem. My first JSON validation CHECK silently accepted a document with no price, because a CHECK that evaluates to NULL passes. In the flash-sale test, the CHECK (stock >= 0) constraint did not stop 40 units being oversold, because each concurrent buyer wrote a valid positive number. Generating realistic test data also took several attempts, as PostgreSQL evaluated some random() subqueries only once until they referenced the outer row.

Lessons learned. An index existing does not mean it is used, and a constraint existing does not mean concurrent writes are safe: both must be tested under realistic load. Materialised views and replicas move cost rather than remove it, so freshness and consistency have to be chosen per use case. Replication and sharding solve different problems, and the choice of shard key decides which queries stay cheap. I now read EXPLAIN ANALYZE output first instead of guessing.

5. References

- Elmasri, R. and Navathe, S. B. (2016) Fundamentals of Database Systems. 7th edn. Pearson.
- Kleppmann, M. (2017) Designing Data-Intensive Applications. O'Reilly Media.
- PostgreSQL Global Development Group (2024) PostgreSQL 16 Documentation (chapters on performance tips, MVCC/concurrency control, high availability and replication, postgres_fdw, JSON types). postgresql.org/docs/16.
- Redis Ltd. Redis documentation: data types, EXPIRE, and programmability with Lua (EVAL). redis.io/docs.
- MongoDB Inc. MongoDB Manual: CRUD operations, aggregation pipeline, data modelling. mongodb.com/docs/manual.
- Federal Republic of Nigeria (2023) Nigeria Data Protection Act, 2023.
- Libraries: psycopg2, redis-py, PyMongo, mongomock, matplotlib, Graphviz. All data in the portfolio is synthetic and generated by the repository scripts.

6. AI use declaration

Tool: Claude (Anthropic), used through the Claude app. **Used for:** drafting the SQL and Python scripts, generating the synthetic data set, running the experiments in a sandboxed environment, and drafting the README files and this documentation. **Review and verification:** I re-ran every script against my own PostgreSQL and Redis installation, checked that the outputs, plans and figures in the evidence folders match my runs, read each README against the results, and edited the explanations so they reflect my own understanding. I can explain and reproduce every component of the portfolio.